

EXAMEN DE CONCURRENCIA Y PARALELISMO

Bloque II: Paralelismo

APELLIDOS: _____ NOMBRE: _____

Convocatoria extraordinaria

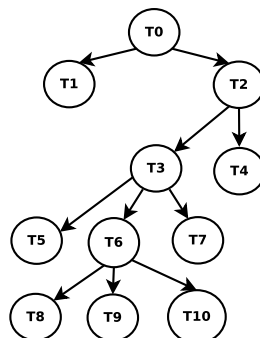
11 de julio de 2022

AVISO: No mezcléis en el mismo folio preguntas del bloque de concurrencia y del bloque de paralelismo. Las respuestas a cada bloque se entregan por separado.

- [2,5p] 1. **Conceptos de paralelismo.** Una empresa ha desarrollado un conjunto de aplicaciones bioinformáticas pensadas para ser ejecutadas en forma de grafo. En todas las ejecuciones se empieza leyendo un genoma (cadena de caracteres A,C,G,T) de un fichero de texto y luego se pueden ejecutar varias aplicaciones sobre él, algunas de las cuales pueden ser realizadas de forma simultánea y otras no (debido a tener dependencias entre ellas). Para cada ejecución será el usuario el que elegirá qué aplicaciones se ejecutan y cuáles no, generando un grafo de tareas concreto para sus análisis.

Contesta DE FORMA RAZONADA las siguientes preguntas:

- (a) ¿Cuál es la aceleración máxima que se puede obtener si la primera tarea (lectura del genoma desde fichero de texto) siempre la hace un único proceso, no se puede simultanear con ninguna otra tarea (nada puede empezar hasta que el genoma esté en memoria) y siempre tarda el 5% del tiempo total?[0,5p]
- (b) Un biólogo utiliza un grafo con 11 tareas, con las dependencias representadas en la siguiente figura y, cada una, requiere los siguientes tiempos para un determinado genoma:



- Tareas 0, 4, 9 y 10: 10 minutos.
- Tareas 1, 3, 5, 7 y 8: 20 minutos.
- Tareas 2 y 6: 30 minutos.

¿Cuáles son el Máximo Grado de Concurrencia y el Grado Medio de Concurrencia? [0,5p]

- (c) En el caso de disponer de tres procesos, ¿cuál sería la asignación óptima? [0,5p]
- (d) ¿Cuáles son el speedup y la eficiencia de la asignación propuesta en el apartado anterior? [0,5p]
- (e) ¿Se consigue alguna ganancia real de tiempo al usar tres procesos, comparado con usar solo dos? Y si tuviésemos más de tres procesos disponibles, ¿sería interesante usar más? [0,5p]

- [2,5p] 2. **Diseño de algoritmos paralelos.** Una de las aplicaciones de la librería presentada en el ejercicio anterior consiste en comparar dos secuencias de ADN (cadenas de caracteres) y proporcionar como resultado el número de posiciones en las que ambas secuencias tienen el mismo valor. A continuación, presentamos el código de la parte principal de dicha aplicación, que recibe como parámetro de entrada la longitud L de las secuencias (ambas deben tener la misma longitud).

```
char *seq1 = (char *)malloc(sizeof(char) * L);
char *seq2 = (char *)malloc(sizeof(char) * L);
leeDeDisco(seq1, seq2, L);
int numIgual = 0;

for(int i = 0; i < L; i++)
    if(seq1[i] == seq2[i])
        numIgual++;

escribeResultado(numIgual);
```

Como estas secuencias pueden ser muy grandes, se quiere paralelizar esa aplicación concreta, teniendo en cuenta que solo uno de los procesos tiene acceso a disco y pantalla. Por tanto, este será el único proceso que puede ejecutar las funciones `leeDeDisco` y `escribeResultado`. Contesta DE FORMA RAZONADA las siguientes preguntas:

- (a) Indica el tipo de descomposición y asignación que usarías para paralelizar esta aplicación. [0,5p]
- (b) Escribe el código MPI asociado a la solución que has indicado en el apartado anterior, usando colectivas siempre que se pueda (ver la tabla del final con la sintaxis de las rutinas MPI). Asume que L es múltiplo del número de procesos. [0,75p]
- (c) Asume que los procesos se encuentran conectados por una red en topología de anillo, donde cada proceso puede comunicarse directamente con otros dos (por ejemplo, el Proceso 1 está conectado con el 0 y con el 2). Estas conexiones son bidireccionales. Explica, DE PALABRA, cómo realizarías una colectiva `MPI_Bcast` en dicha red si tiene como proceso raíz el 0. ¿Cuántos pasos se necesitarían para completar el *broadcast* sobre sobre $numP$ procesos? [0,5p]
- (d) Escribe con código MPI (usando funciones `MPI_Send` y `MPI_Recv`) la implementación de la colectiva del apartado anterior cuando recibe como entrada un array de caracteres A de un tamaño arbitrario L . [0,75]

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype,
             int dest, int tag, MPI_Comm comm)
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
             int source, int tag, MPI_Comm comm, MPI_Status *status)
int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype,
             int root, MPI_Comm comm)
int MPI_Scatter(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
              void *recvbuf, int recvcnt, MPI_Datatype recvtype,
              int root, MPI_Comm comm)
int MPI_Gather(void *sendbuf, int sendcnt, MPI_Datatype sendtype,
              void *recvbuf, int recvcnt, MPI_Datatype recvtype,
              int root, MPI_Comm comm)
int MPI_Allreduce(void *sendbuf, void *recvbuf, int count,
                 MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
              MPI_Datatype datatype, MPI_Op op,
              int root, MPI_Comm comm)
```

SOLUCIÓN

1. (a) La Ley de Amdahl la aceleración máxima es:

$$\frac{T_s + T_p}{T_s}$$

En este caso, la parte intrínsecamente secuencial es el 0.05 del tiempo. Lo que quiere decir que la aceleración máxima es:

$$\frac{0.05 + 0.95}{0.05} = 20$$

- (b) El Máximo Grado de Concurrencia sería 7, ya que todos los extremos del final del árbol se podrían ejecutar a la vez.

Para calcular el Grado Medio de Concurrencia necesitamos determinar el camino crítico: T0-T2-T3-T6-T8 (longitud 110 minutos). Todas las tareas en secuencial tardan 200 minutos, así que el Grado Medio de Concurrencia es:

$$\frac{200}{110} = 1.82$$

- (c) Podría haber varias asignaciones óptimas. Una con la que el tiempo paralelo es igual que el Grado Medio de Concurrencia (y, por tanto, no es mejorable) sería:

- Proceso 0: T0, T2, T3, T6, T8
- Proceso 1: T1, T4, T5, T9
- Proceso 2: T7, T10

En la siguiente tabla se puede ver que esta asignación necesita 110 minutos (cada fragmento en horizontal representa 10 minutos):

Proceso 0	T0	T2	T2	T2	T3	T3	T6	T6	T6	T8	T8
Proceso 1		T1	T1		T4		T5	T5		T9	
Proceso 2							T7	T7		T10	

- (d) El tiempo paralelo se computa desde que se inicia el primer proceso hasta que termina el último. En este caso es la carga del Proceso 0 (110 minutos).

$$S = \frac{200}{110} = 1.82$$

$$Eff = \frac{1.82}{3} = 0.61$$

- (e) Si no tuviésemos el tercer proceso no podríamos trabajar en paralelo a la vez con T5, T6 y T7, así que sí compensa utilizarlo. Sin embargo, por culpa de las dependencias, aunque dispongamos de más procesos no hay manera de meter una cuarta fase en paralelo en ningún punto del pipeline.
2. (a) Sería una descomposición de dominio con una tarea por cada posición de las secuencias ya que en todas ellas se realiza la misma operación (comparar si son iguales y, si es necesario, sumar uno al contador). La asignación más óptima sería una estática ya que la carga de trabajo por tarea es homogénea. Daría igual una cíclica o una bloque pero, por ser más sencilla de implementar, lo lógico es usar una bloque.

```

char *seq1, *seq2;
int numIgual, resParcial;

if(rank == 0){
    seq1 = (char *)malloc(sizeof(char) * L);
    seq2 = (char *)malloc(sizeof(char) * L);
    leeDeDisco(seq1, seq2, L);
} else {
    seq1 = (char *)malloc(sizeof(char) * L/numP);
    seq2 = (char *)malloc(sizeof(char) * L/numP);
}

resParcial = 0;

MPI_Scatter(seq1, L/numP, MPI_CHAR,
    rank ? seq1 : MPI_IN_PLACE, L/numP, MPI_CHAR, 0, MPI_COMM_WORLD);
MPI_Scatter(seq2, L/numP, MPI_CHAR,
    rank ? seq2 : MPI_IN_PLACE, L/numP, MPI_CHAR, 0, MPI_COMM_WORLD);

for(int i = 0; i < L/numP; i++)
    if(seq1[i] == seq2[i])
        resParcial++;

MPI_Reduce(&resParcial, &numIgual, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);

if(rank == 0)
    escribeResultado(numIgual);

```

- (c) Inicialmente el array estaría en la memoria del Proceso 0. Como tiene dos uniones bidireccionales aprovechará para mandar a la vez el array por el camino del Proceso 1 y por el camino del proceso $numP - 1$. Cada proceso, una vez que recibe, pasa el array al siguiente hasta que todos lo tengan. Hay que tener en cuenta que uno de los caminos puede ser más largo que el otro según si el número de procesos es múltiplo de dos o no.

El número de pasos será $\frac{numP}{2}$.

```

int soyFinal = 0;
if((rank == numP/2) || (rank == numP/2+1))
    soyFinal = 1;

int caminoAscendente = 0;
if(rank <= numP/2)
    caminoAscendente = 1;

if(rank == 0) {
    MPI_Send(A, L, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
    MPI_Send(A, L, MPI_CHAR, numP-1, 0, MPI_COMM_WORLD);
} else {
    MPI_Recv(A, L, MPI_CHAR, MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, null);
    if(soyFinal == 0){
        if(caminoAscendente == 1){
            MPI_Send(A, L, MPI_CHAR, rank+1, 0, MPI_COMM_WORLD);
        } else {
            MPI_Send(A, L, MPI_CHAR, rank-1, 0, MPI_COMM_WORLD);
        }
    }
}
}

```